

Т3 модернизации

Process-Oriented World Model — расширение существующей Машины Эпистемологии

Принцип:	Модернизация, а не замена. Всё, что работает — сохраняется.
Фундамент:	ADR-O-322 «Машина Эпистемологии» (5 ненарушимых инвариантов)
Опирается на:	EventMemory, AffordanceVector, BeliefState, WorldTopologyProvider, DynamicAffordanceField, ObservationRela
Добавляет:	Process/Transformation, WorldGraph, NPCKnowledge, Planner, композиц. Affordance
Дата:	2026-07-16

Тезис модернизации

Проект уже построен как «Машина Эпистемологии» (ADR-O-322) с 5 ненарушаемыми инвариантами, включая запрет телепатии. Это **нельзя трогать** — это фундамент. Однако в нём есть пробел: нет первоклассной сущности «процесс, меняющий мир ». Сейчас мир меняется через разрозненные каналы (EventDTO, SceneChange, StateApplicator, LifeEngine.tick), что не позволяет NPC строить планы как пути в графе возможностей мира. Предлагается ввести слой **Process** поверх существующих Manifestation/EventDTO, а также **WorldGraph** и **NPCKnowledge** как явные сущности. Существующие системы продолжают работать; новые слои только добавляются.

Карта: что есть, что добавляется

Слой	Статус	Где живёт	Что делает
ADR-O-322 Epistemology Machine	ЕСТЬ	docs/APXИТЕКТУРНЫЙ УСТАВ_ENIGMA-1731-00_CAUSAL_CONTRACT_v2.0.md	Постановка: невозрастание истины, запрет кау...
Manifestation + ObservationRelation	ЕСТЬ	domain/world_projection.py services/perception/perception_projector.py	Reality → Manifestation → ObservationRelation →
EventMemory с known_by/hidden_from	ЕСТЬ	models/npc_state.py:200-236	Хранит эпистемические поля: is_secret, known_by
AffordanceVector (физический)	ЕСТЬ	domain/motion_core.py:27	can_stand, can_vault, can_climb, can_conceal, can_p
DynamicAffordanceField (стигмергия)	ЕСТЬ	services/spatial/world_topology_provider.py:16	Hard Overrides + Soft Traces — динамические деф
BeliefState + BeliefTransitionEngine + BeliefCrystallizationEngine + CrystallizedBeliefStore	ЕСТЬ (сломано в runtime, см. отчёт №1)	services/npc/belief_transition_engine.py services/npc/belief_crystallization_engine.py services/npc/crystallized_belief_store.py	Эпистемическая цепочка для убеждений NPC. Ко
TruthState / ObservationLog / PlayerBeliefModel / EvaluationEngine	ОПИСАНО в ТЗ, НЕ реализовано	docs/.../ТЕХНИЧЕСКОЕ ЗАДАНИЕ Мониторинг Фактов (16 секрето...	Мониторинг Фактов (16 секретов).
Process / Transformation	НОВОЕ	services/world_model/process.py (новый файл)	Первоклассная сущность «процесс, меняющий ми
WorldGraph	НОВОЕ	services/world_model/world_graph.py (новый файл)	Граф состояний мира. Узлы = состояния, рёбра =
NPCKnowledge	НОВОЕ	services/world_model/npc_knowledge.py (новый файл)	Субъективное знание NPC о мире. last_verified_tic
Planner	НОВОЕ	services/planner/world_planner.py (новый файл)	Поиск путей в WorldGraph с учётом NPCKnowled
Композиционный Affordance	НОВОЕ	models/world/affordance.py (новый файл)	(domain, attribute, value, props) — открытая онтоло

1. Существующий фундамент (НЕ ТРОГАТЬ)

1.1 ADR-O-322 — 5 ненарушимых инвариантов

УЖЕ ЕСТЬ: АРХИТЕКТУРНЫЙ_УСТАВ_ENIGMA.md:731-741. Система официально является «Машиной Эпистемологии». Паттерн: State → Manifestation → ObservationRelation → Signal → Fact → Hypothesis → Belief → Consumers.

5 инвариантов:

1. Закон невозрастания истины — слои не увеличивают истину, только теряют/преобразуют
2. Запрет каузального возврата — Inference не меняет Reality
3. Изоляция потребителей — DM/UI/CDS получают только эпистемическое представление
4. Реляционная сущность — ObservationRelation отдельный DTO, не поля объектов
5. Manifestation — единственный мост, immutable

Тест на телепатию уже существует: test_no_telepathy_in_ui_observation (00_CAUSAL_CONTRACT_v2.0.md:405).

1.2 EventMemory — уже хранит эпистемические поля

УЖЕ ЕСТЬ: models/npc_state.py:200-236. EventMemory — frozen dataclass, хранит:

- is_secret: bool
- known_by: Tuple[str, ...] ← кто знает
- hidden_from: Tuple[str, ...] ← от кого скрыто
- accessibility: float (0..1, падает со временем)
- clarity, confidence, decay_rate
- actor_id: str ← кто совершил
- tags: Tuple[str, ...]

Это уже эпистемический барьер на уровне памяти. NPC знает только то, что записано в его EventMemory с известным ему known_by.

Вывод: запрет телепатии формально соблюдается. Но он работает только на уровне памяти — NPC не может вспомнить то, чего не знал. Проблема в том, что DecisionHub и пространственная логика (life_engine._resolve_position) обращаются к миру напрямую, а не через эпистемический фильтр. Это и есть пробел, который закрывает данное ТЗ.

1.3 AffordanceVector — физический слой уже есть

УЖЕ ЕСТЬ: domain/motion_core.py:27. AffordanceVector — frozen dataclass:

- can_stand, can_vault, can_climb, can_conceal, can_pass — физические возможности
- surface_grip, drag_coefficient — кинематическое сопротивление
- light_level, thermal_mass, exposure, mana_resonance — сенсорные градиенты

Это физический слой affordances. Его НЕ трогаем. Новая композиционная Affordance строится поверх — как расширение для социального/торгового/духовного слоёв, которых сейчас нет.

1.4 DynamicAffordanceField — стигмергия уже есть

УЖЕ ЕСТЬ: services/spatial/world_topology_provider.py:16. DynamicAffordanceField — уже двухслойная система:

- Hard Overrides: структурные деформации (Absolute Override)
- Soft Traces: поведенческие следы (накопление и decay)

Это и есть часть Process-слоя: мир меняется от действий NPC, оставляя следы. Расширяем, не заменяем.

1.5 BeliefState + BeliefTransitionEngine + CrystallizationEngine

УЖЕ ЕСТЬ: Полная цепочка L2.5 убеждений уже реализована:

- models/npc/beliefs.py — BeliefState + BeliefFragment + BeliefType enum
- services/npc/belief_transition_engine.py — reactive writer (R7)
- services/npc/belief_crystallization_engine.py — pattern-based writer (R8)
- services/npc/crystallized_belief_store.py — in-memory хранилище
- services/memory/belief_aggregator.py — Evidence → BeliefFragment
- services/memory/evidence_mapper.py — EventMemory → Evidence

По отчёту №1: код работает, в runtime не срабатывает из-за upstream-багов memory write path. Это чинится отдельно (R1-R3 из отчёта №1).

1.6 TruthState из ТЗ «Секреты Люси» — описано, не реализовано

УЖЕ ЕСТЬ: docs/.../ТЕХНИЧЕСКОЕ ЗАДАНИЕ Миниигра «Таверна Серебряный Волк».md. 16 секретов, 6 NPC, 20 каузальных связей, 3 кластера.

TruthState / ObservationLog / PlayerBeliefModel / EvaluationEngine — контракты описаны, кода нет. Это точка роста для player-side эпистемики. Данное ТЗ не закрывает этот пробел, но формирует NPC-side фундамент, поверх которого позже строится player-side.

2. Process — первоклассная сущность

Сейчас мир меняется через 5+ разрозненных каналов: EventDTO, SceneChange, StateApplicator, LifeEngine.tick, combat_subscriber, transaction_engine. Каждый делает свою часть, нет единого представления «процесс, меняющий мир».

2.1 Определение Process

ПРОЦЕСС: Process — это описание того, как одно состояние мира трансформируется в другое. Process не имеет «исполнителя» — это может быть действие NPC, природное явление, время суток, предмет, животное.

Process описывает ТОЛЬКО трансформацию. Конкретный источник — отдельная сущность (ProcessSource), может быть NPC / Location / Item / Animal / TimeOfDay / Weather.

```
# backend/app/models/world/process.py (НОВЫЙ ФАЙЛ)
from dataclasses import dataclass, field
from typing import Optional, Tuple, Dict, Any

@dataclass(frozen=True)
class WorldStateDelta:
    """Дельта состояния мира, которую применяет Process."""
    domain: str # 'resource', 'environment', 'social', 'item', ...
    attribute: str # 'alcohol', 'temperature', 'crowd_density', ...
    delta: Any # +1, -1, 'broken→functional', 'unknown→known'
    target_ref: str # что меняется: 'location:tavern:bar_area' / 'npc:orm' / 'item:sword'

@dataclass(frozen=True)
class Process:
    """Абстрактная трансформация состояния мира.

    НЕ знает кто/что её запускает. Знает только:
    - какие WorldStateDelta применяет
    - какие WorldState (предикаты) требует для запуска
    - сколько длится
    """
    process_id: str # 'item_repair', 'alcohol_serving', 'rain_falls'
    applies_deltas: Tuple[WorldStateDelta, ...]
    requires_world_state: Tuple[WorldPredicate, ...] # (domain, attribute, value)
    duration_ticks: int = 1
    side_effects: Tuple[str, ...] = () # event_type для EventDTO

@dataclass(frozen=True)
class ProcessSource:
    """Конкретный источник Process. Может быть кем/чем угодно."""
    source_id: str # 'blacksmith_orm', 'forge_orm', 'rain_system', 'time'
    source_kind: str # 'npc' | 'location' | 'item' | 'environment' | 'time'
    available_processes: Tuple[str, ...] # process_ids, которые может запускать
    requires_provider_state: Tuple[WorldPredicate, ...] = ()
    # HET provider_type — это просто источник
```

2.2 Примеры Process

```
# Пример 1: починка предмета (через NPC-кузнеца)
process_id: 'item_repair_via_blacksmith'
applies_deltas:
- { domain: 'item', attribute: 'state', delta: 'broken→functional',
  target_ref: 'item:{target_item}' }
- { domain: 'npc', attribute: 'gold', delta: -5,
  target_ref: 'npc:{customer}' }
- { domain: 'relationship', attribute: 'trust', delta: +0.05,
  target_ref: 'npc:{customer}→npc:{blacksmith}' }
requires_world_state:
- { domain: 'infrastructure', attribute: 'metalwork', value: 'present' }
- { domain: 'npc', attribute: 'health', value: '>0', target: '{blacksmith}' }
duration_ticks: 5

# Пример 2: подача алкоголя (через трактирщика)
process_id: 'alcohol_serving'
applies_deltas:
- { domain: 'npc', attribute: 'intoxication', delta: +0.2, target: '{customer}' }
- { domain: 'npc', attribute: 'gold', delta: -1, target: '{customer}' }
- { domain: 'npc', attribute: 'gold', delta: +1, target: '{keeper}' }
requires_world_state:
- { domain: 'resource', attribute: 'alcohol', value: 'present' }
duration_ticks: 1

# Пример 3: дождь наполняет реку (БЕЗ ИСПОЛНИТЕЛЯ)
process_id: 'rain_fills_river'
applies_deltas:
- { domain: 'environment', attribute: 'water_level', delta: +0.3,
  target_ref: 'location:river' }
requires_world_state:
- { domain: 'weather', attribute: 'rain_intensity', value: '>0.5' }
duration_ticks: 10

# Пример 4: ночь делает местность тёмной (БЕЗ ИСПОЛНИТЕЛЯ)
process_id: 'nightfall_darkens_area'
applies_deltas:
- { domain: 'environment', attribute: 'light_level', delta: '→low' }
requires_world_state:
- { domain: 'time', attribute: 'hour', value: 'in [22, 6]' }
duration_ticks: 1

# Пример 5: пожар уничтожает склад (БЕЗ ИСПОЛНИТЕЛЯ)
process_id: 'fire_destroys_storage'
applies_deltas:
- { domain: 'location', attribute: 'structural_integrity', delta: '→0',
  target_ref: 'location:storage_1' }
- { domain: 'item', attribute: 'state', delta: '→destroyed' }
requires_world_state:
- { domain: 'environment', attribute: 'fire', value: 'present' }
duration_ticks: 50
```

2.3 Связь с существующими системами

Process НЕ заменяет EventDTO, SceneChange, StateApplicator. Process — это **декларативное описание** трансформации. При запуске Process генерирует те же самые EventDTO и SceneChange, что и раньше — просто через единую точку.

```
# backend/app/services/world_model/process_executor.py (НОВЫЙ)
from app.domain.events import EventDTO
from app.services.scene_change import SceneChange

class ProcessExecutor:
```

```
"""Превращает Process в конкретные EventDTO + SceneChange.

ИСПОЛЬЗУЕТ существующие каналы – не заменяет.
"""
def execute(self, process: Process, source: ProcessSource,
context: Dict) -> List[EventDTO | SceneChange]:
    events = []
    for delta in process.applies_deltas:
        # 1. Применить через StateApplicator (существующий канал)
        self._state_applicator.apply_delta(delta, context)
        # 2. Сгенерировать EventDTO для подписчиков (существующий канал)
        events.append(self._make_event_dto(process, delta, source))
        # 3. При необходимости – SceneChange для фронта (существующий канал)
        if self._needs_scene_change(delta):
            events.append(self._make_scene_change(delta, context))
    return events
```

3. WorldGraph — граф состояний мира

Сейчас WorldState (services/simulation/world_state.py) — это токен-бюджетный срез для DM-контекста. Это не граф, нет рёбер, нет поиска путей. Planner не может работать без графа.

ДОБАВИТЬ: WorldGraph — граф, где узлы = снимки состояния мира (snapshot), рёбра = Process, который может перевести одно состояние в другое. Граф строится лениво: полные узлы не хранятся, хранятся только текущее состояние + available processes (как рёбра-кандидаты).

Planner ищет путь BFS/DFS от текущего состояния к целевому (заданному Goal), с ограничением глубины (max_depth=5) и эпистемическим фильтром (NPCKnowledge).

```
# backend/app/services/world_model/world_graph.py (НОВЫЙ)
from typing import List, Set, Optional, Dict
from app.models.world.process import Process, ProcessSource, WorldStateDelta
from app.models.world.affordance import WorldPredicate

class WorldGraph:
    """Ленивый граф состояний мира.

    Не хранит все узлы — их бесконечно. Хранит:
    - текущее состояние мира (WorldSnapshot)
    - реестр всех известных Process и ProcessSource
    - методы поиска путей от текущего к целевому состоянию
    """

    def __init__(self, process_registry, source_registry, world_snapshot):
        self._processes = process_registry # все Process
        self._sources = source_registry # все ProcessSource
        self._snapshot = world_snapshot # текущее состояние

    def find_paths(
        self,
        from_snapshot: WorldSnapshot,
        target_predicates: Set[WorldPredicate],
        max_depth: int = 5,
        filter_by_npc_knowledge: Optional['NPCKnowledge'] = None,
    ) -> List[Plan]:
        """BFS поиск путей от from_snapshot к состоянию, удовлетворяющему target_predicates.

        Эпистемический фильтр: если filter_by_npc_knowledge задан, отсекает рёбра,
        использующие ProcessSource, которых NPC не знает.
        """

        # 1. Проверить, не удовлетворяет ли уже from_snapshot целевому
        if self._satisfies(from_snapshot, target_predicates):
            return [Plan(steps=[])] # уже достигнуто

        # 2. BFS
        visited = set()
        queue = [(from_snapshot, [])]
        plans = []

        while queue and len(plans) < 10: # максимум 10 кандидатов
            current, path = queue.pop(0)
            if len(path) >= max_depth:
                continue

            snapshot_key = self._snapshot_key(current)
            if snapshot_key in visited:
                continue
            visited.add(snapshot_key)
```



```
# 3. Найти все Process, чьи requires_world_state удовлетворены
for process in self._processes.all():
    if not self._requirements_met(current, process.requires_world_state):
        continue

# 4. Найти все ProcessSource, которые могут запустить этот process
for source in self._sources.for_process(process.process_id):
    # Эпистемический фильтр
    if filter_by_npc_knowledge:
        if not filter_by_npc_knowledge.knows_source(source.source_id):
            continue

# 5. Применить process – получить новый снимок
new_snapshot = self._apply_process(current, process, source)
new_path = path + [PlanStep(process, source, new_snapshot)]

if self._satisfies(new_snapshot, target_predicates):
    plans.append(Plan(steps=new_path))
else:
    queue.append((new_snapshot, new_path))

return plans
```

3.1 Кэширование

Planner кэширует **результаты поиска** (найденные провайдеры, маршруты, оценки), но НЕ кэширует Plan. Plan — короткоживущий, перевычисляется каждые N тиков или при значимом изменении мира. Это согласуется с вашим требованием: мир меняется — план должен пересчитаться.

4. NPCKnowledge — эпистемический фильтр для Planner'a

EventMemory уже хранит known_by/hidden_from, но это фильтр на уровне памяти. Planner'у нужен фильтр на уровне «**что NPC знает о ProcessSource**» — иначе NPC будет телепатически знать о существовании кузнеца в городе, где никогда не был.

ДОБАВИТЬ: NPCKnowledge — per-NPC реестр субъективного знания о ProcessSource. Хранит не confidence (которая «гниёт» сама по себе), а last_verified_tick + world_revision. Устаревает мир, а не память.

Когда мир существенно меняется (Орм умер, кузница сгорела, караван уехал), knowledge автоматически становится потенциально устаревшим — потому что world_revision источника не совпадает с last_verified_tick NPC.

```
# backend/app/services/world_model/npc_knowledge.py (НОВЫЙ)
from dataclasses import dataclass, field
from typing import Dict, Set, Optional
from enum import Enum

class KnowledgeSource(Enum):
    MET_DIRECTLY = 'met_directly' # был в той же локации
    HEARD_FROM = 'heard_from' # другой NPC рассказал
    RUMOR_HEARD = 'rumor_heard' # подслушал слух
    OBSERVED_USE = 'observed_use' # видел, как использовали
    EXPLICIT_INQUIRY = 'explicit_inquiry' # целенаправленно спросил
    PLAYER_TOLD = 'player_told' # игрок сообщил

@dataclass
class KnowledgeEntry:
    """Одна запись о знании NPC о ProcessSource."""
    source_id: str # 'blacksmith_orm', 'forge_orm', 'church'
    known_processes: Set[str] # process_ids, которые NPC знает у этого source
    origin: KnowledgeSource # откуда знает
    origin_chain: tuple # цепочка происхождения (для ложных слухов)
    # (source_npc, target_npc, tick) — кто кому рассказал, и когда
    last_verified_tick: int # когда NPC последний раз проверял лично
    world_revision_at_verification: int # ревизия мира в момент проверки
    believed_location: Optional[str] = None # где NPC думает, что source находится
    is_false: bool = False # ложный слух (т player может узнать)

class NPCKnowledge:
    """Per-NPC субъективное знание о мире."""
    def __init__(self, npc_id: str):
        self.npc_id = npc_id
        self._entries: Dict[str, KnowledgeEntry] = {}

    def knows_source(self, source_id: str) -> bool:
        entry = self._entries.get(source_id)
        if entry is None:
            return False
        # Проверить, не устарело ли знание
        current_revision = self.world.get_source_revision(source_id)
        if current_revision > entry.world_revision_at_verification:
            # Мир изменился с момента проверки — знание потенциально устарело
            # Planner может использовать, но с пометкой 'unverified'
            return True # всё ещё «знает», но confidence ниже
        return True

    def knows_process(self, source_id: str, process_id: str) -> bool:
        entry = self._entries.get(source_id)
```

```

return entry is not None and process_id in entry.known_processes

def add_knowledge(self, source_id: str, process_id: str,
origin: KnowledgeSource, origin_chain: tuple,
current_tick: int, world_revision: int):
if source_id not in self._entries:
self._entries[source_id] = KnowledgeEntry(
source_id=source_id,
known_processes=set(),
origin=origin,
origin_chain=origin_chain,
last_verified_tick=current_tick,
world_revision_at_verification=world_revision
)
self._entries[source_id].known_processes.add(process_id)

def get_unverified_entries(self, current_revision: int) -> List[KnowledgeEntry]:
"""Записи, которые могут быть устаревшими – NPC стоит проверить."""
return [
e for e in self._entries.values()
if current_revision > e.world_revision_at_verification
]

```

4.1 Цепочка происхождения слуха

Ложные слухи не генерируются случайно. Они имеют **цепочку происхождения**:

```

# Пример: Shadow соврал Борко, Борко рассказал Люсе
maid_lusya.knowledge._entries['tavern_keeper_tornin']:
known_processes: {'alcohol_serving', 'lodging_booking'}
origin: HEARD_FROM
origin_chain: (
('shadow', 'borko', tick=42), # Shadow рассказал Борко
('borko', 'lusya', tick=58), # Борко рассказал Люсе
)
last_verified_tick: 58
world_revision_at_verification: 7
is_false: True # слух ложный – но Люся не знает

# Если игрок позже раскроет ложь (через шантаж Тени):
# → maid_lusya.knowledge.mark_false('tavern_keeper_tornin', revealed_by='player')
# → в player_belief_model добавляется убеждение «Тень лжец»
# → в npc_relationships[lusya][shadow].trust -= 20

```

Это связывает NPCKnowledge с ТЗ «Секреты Люси» — ложные слухи становятся частью каузального графа секретов, а не случайным шумом.

5. Композиционный Affordance (открытая онтология)

Существующий AffordanceVector (motion_core.py) — физический, enum-based. Его не трогаем. Новый композиционный Affordance — параллельный слой для социального/торгового/духовного, с открытой онтологией (domain, attribute, value).

ДОБАВИТЬ: Affordance — это предикат над миром: (domain, attribute, value, props). Domain и attribute — открытые идентификаторы (не enum). Появление нового домена НЕ архитектурное событие — просто новая запись в каталоге.

```
# backend/app/models/world/affordance.py (НОВЫЙ)
from dataclasses import dataclass, field
from typing import Any, Tuple, FrozenSet

@dataclass(frozen=True)
class AffordancePredicate:
    """Композиционный предикат возможности мира.

    НЕ enum. Открытая онтология — domain и attribute могут быть любыми строками.
    Planner ищет по предикату: world.find_where(domain='resource', attribute='alcohol').
    """
    domain: str # 'resource', 'environment', 'social', 'infrastructure', ...
    attribute: str # 'alcohol', 'temperature', 'crowd_density', 'metalwork', ...
    value: Any # 'present', 'warm', 'high', True, 5, ...
    props: FrozenSet[str] = frozenset() # необязательные модификаторы

    def matches(self, other: 'AffordancePredicate') -> bool:
        """Совпадение с допуском: value может быть диапазоном или предикатом."""
        if self.domain != other.domain: return False
        if self.attribute != other.attribute: return False
        # value может быть строкой '>5', 'present', или точным значением
        return self._value_matches(self.value, other.value)

# Примеры (НЕ enum — просто предикаты):
ALCOHOL_AVAILABLE = AffordancePredicate(
    domain='resource', attribute='alcohol', value='present',
    props=frozenset({'consumable', 'intoxicating'})
)
WARM_AREA = AffordancePredicate(
    domain='environment', attribute='temperature', value='warm'
)
CONCEALMENT = AffordancePredicate(
    domain='cover', attribute='visual', value='available',
    props=frozenset({'partial'})
)
CROWD_PRESENT = AffordancePredicate(
    domain='social', attribute='crowd_density', value='high'
)
AUTHORITY_PRESENT = AffordancePredicate(
    domain='authority', attribute='presence', value='armed',
    props=frozenset({'corruptible'})
)

# Расширение словаря = добавить новый предикат. Старые не меняются.
# Через полгода появятся:
# HORSE_AVAILABLE = AffordancePredicate(domain='transport', attribute='mount',
# value='horse')
# FESTIVAL_ACTIVE = AffordancePredicate(domain='social', attribute='event_type',
# value='wedding')
# Ничего менять в существующем коде не нужно.
```

5.1 Каталог доменов (стартовый, открытый)

Файл `config/world/ontology/domains.yaml` содержит стартовый набор доменов и атрибутов. Это НЕ закрытый enum — просто документация. Появление нового домена = новая запись в YAML, без правок кода.

```
# config/world/ontology/domains.yaml
domains:
  resource: # что есть в мире физически
  attributes: [alcohol, food, iron, wood, water, gold, mana]
  environment: # свойства среды
  attributes: [temperature, light_level, weather, mana_level, noise]
  cover: # укрытия
  attributes: [visual, acoustic, magical]
  social: # социальные свойства
  attributes: [crowd_density, social_class, faction_dominant]
  authority: # власть
  attributes: [presence, type, jurisdiction]
  infrastructure: # физическая инфраструктура
  attributes: [metalwork, woodwork, kitchen, commerce, sleeping, sacred]
  commerce: # торговая инфраструктура
  attributes: [market_venue, currency_exchange, storage]
  spiritual: # духовные свойства
  attributes: [sanctuary, confession_space, meditation_space, sacred_space]
  safety: # безопасность
  attributes: [danger_level, refuge_available, faction_territory]
  time: # время
  attributes: [hour, day_phase, season]
  faction: # фракции
  attributes: [controlled_by, influence_level, presence]
# Открыто для расширения – добавляйте новые домены без правок кода
```

5.2 Связь с AffordanceVector

Существующий `AffordanceVector` (`motion_core.py:27`) описывает физические возможности среды для движения. Новый `AffordancePredicate` описывает **социальные, торговые, духовные** возможности. Это два разных слоя, не конфликтующих:

```
# Существующий физический слой (НЕ ТРОГАЕМ):
class AffordanceVector:
  can_stand: float
  can_vault: float
  can_climb: float
  can_conceal: float
# ... остаётся для кинематики

# Новый композиционный слой (ДОБАВЛЯЕМ):
class AffordancePredicate:
  domain: str
  attribute: str
  value: Any
# ... для социальных, торговых, духовных возможностей

# Bridge: AffordanceVector → AffordancePredicate (только для чтения)
# Позволяет Planner'у использовать оба слоя единообразно
def vector_to_predicates(vec: AffordanceVector) -> List[AffordancePredicate]:
  preds = []
  if vec.can_conceal > 0.5:
    preds.append(AffordancePredicate('cover', 'visual', 'available'))
  if vec.light_level < 0.3:
    preds.append(AffordancePredicate('environment', 'light_level', 'low'))
  return preds
```

6. Planner — поиск путей в графе мира

Planner — главное новое поведение. Заменяет прямые вызовы DecisionHub для proactive-целей. DecisionHub остаётся для reactive-решений (что делать прямо сейчас в ответ на событие).

ДОБАВИТЬ: Planner строит Plan как последовательность PlanStep, каждый из которых:

- запускает Process через ProcessSource
- меняет состояние мира
- фильтруется по NPCKnowledge (NPC не знает — не использует)
- если нужного source NPC не знает — Planner генерирует план «узнать»

```
# backend/app/services/planner/world_planner.py (НОВЫЙ)
from typing import Optional, List
from app.models.world.process import Process, ProcessSource
from app.services.world_model.world_graph import WorldGraph, Plan, PlanStep
from app.services.world_model.npc_knowledge import NPCKnowledge

class WorldPlanner:
    """Поиск путей в графе мира с эпистемическим фильтром.

    Заменяет прямые вызовы DecisionHub для proactive-целей.
    DecisionHub остаётся для reactive-решений.
    """
    def __init__(self, world_graph: WorldGraph, knowledge_registry):
        self._graph = world_graph
        self._knowledge_registry = knowledge_registry # per-NPC NPCKnowledge

    def plan(self, npc_id: str, goal: 'Goal', current_tick: int) -> Optional[Plan]:
        """Построить план достижения goal для данного NPC."""
        knowledge = self._knowledge_registry.get(npc_id)
        target_predicates = goal.to_world_predicates()
        current_snapshot = self._graph.current_snapshot()

        # 1. Поиск путей через известных провайдеров
        plans = self._graph.find_paths(
            from_snapshot=current_snapshot,
            target_predicates=target_predicates,
            max_depth=5,
            filter_by_npc_knowledge=knowledge,
        )

        # 2. Если путей нет – попробовать план «узнать нового провайдера»
        if not plans:
            ask_plan = self._build_information_seeking_plan(
                npc_id, goal, knowledge, current_tick
            )
            if ask_plan:
                return ask_plan
            return None # план невозможен

        # 3. Оценить и выбрать лучший
        evaluated = [self._evaluate(p, npc_id) for p in plans]
        evaluated.sort(key=lambda e: e.score, reverse=True)

        if evaluated[0].score < MIN_PLAN_SCORE:
            return None
        return evaluated[0].plan

    def _build_information_seeking_plan(self, npc_id, goal, knowledge, tick):
        """Если NPC не знает подходящего провайдера – спросить знакомого.
```

```

1. Найти NPC с высоким trust, который может знать
2. Подойти к нему
3. Запустить Process 'information_acquisition'
4. После получения знания — обычный plan станет доступен
"""
# NPC, которых данный NPC знает лично
known_npcs = knowledge.get_known_npc_sources()
if not known_npcs:
    return None # совсем никого не знает — тупик

# Выбрать NPC с высоким trust
target_npc = self._pick_most_trusted(known_npcs, npc_id)
if not target_npc:
    return None

# Построить план: APPROACH + DIALOGUE(topic='do_you_know_*')
return Plan(steps=[
    PlanStep(process='approach_npc', source=target_npc),
    PlanStep(process='information_inquiry', source=target_npc,
        params={'topic': goal.required_capability_type}),
])
# После выполнения: knowledge.add_knowledge(...) обновит NPCKnowledge
# На следующем тике обычный plan найдёт путь

```

6.1 Канонический пример

Цель Люси: «починить сломанный меч». Орм ей неизвестен как кузнец (она знает его только как мужчину, с которым спит).

```

Tick 0: Lusya has goal 'repair_sword'
↓
Planner.plan(lusya, goal='repair_sword'):
knowledge.knows_source('blacksmith_orm') = True
knowledge.knows_process('blacksmith_orm', 'item_repair') = FALSE ← не знает!
→ paths through orm FILTERED OUT
→ no direct paths found
→ build_information_seeking_plan:
known_npcs = ['tavern_keeper_tornin', 'blacksmith_orm_as_lover']
trust[tornin] = 0.3, trust[orm_as_lover] = 0.6
target = orm_as_lover (она ему доверяет как любовнику)
plan = [APPROACH orm, DIALOGUE(topic='can_you_repair_sword?')]
↓
Tick 1: Lusya approaches orm, asks
→ ORM confirms: 'yes, I can repair. 5 gold.'
→ NPCKnowledge.add_knowledge('blacksmith_orm', 'item_repair',
    origin=MET_DIRECTLY, tick=1)
↓
Tick 2: Planner.plan(lusya, goal='repair_sword'):
knowledge.knows_process('blacksmith_orm', 'item_repair') = True
→ path found: [MOVE to forge, APPROACH orm, EXECUTE item_repair]
→ score = 0.8 (close, affordable, available now)
↓
Tick 3-7: Execution of Plan
→ ProcessExecutor.execute(item_repair, source=orm, target=lusya_sword)
→ WorldStateDelta applied: sword.state = 'functional', lusya.gold -= 5
→ EventDTO generated (existing channel)
→ SceneChange generated (existing channel)
→ Existing perception pipeline picks it up

```

Если Орм умрёт между Tick 1 и Tick 2 — world_revision кузнеца изменится. NPCKnowledge вернёт knows_source=True, но знает что last_verified_tick устарел. Planner пометит план как 'unverified' и при выполнении получит отказ — сгенерирует новый goal «узнать, что случилось с Ормом».

7. Структура новых файлов (YAML как источник истины)

YAML — единственный источник истины. Никакой автогенерации JSON. При загрузке система строит внутренние индексы, графы и кэши напрямую из YAML.

```
config/world/
├── _catalog.md # человекочитаемый индекс
├── ontology/
│   ├── domains.yaml # открытая онтология доменов/атрибутов
│   └── processes.yaml # каталог Process'ов (~20 абстрактных)
├── sources/ # ProcessSource (без типа)
│   ├── blacksmith_orm.yaml # NPC как источник
│   ├── tavern_keeper_tornin.yaml
│   ├── maid_lusya.yaml
│   ├── guard_borko.yaml
│   ├── merchant_goran.yaml
│   ├── thief_shadow.yaml
│   ├── forge_orm.yaml # локация как источник
│   ├── tavern_silver_wolf.yaml
│   ├── market_square.yaml
│   ├── city_gate.yaml
│   ├── church.yaml
│   ├── forest.yaml
│   ├── anvil_station.yaml # предмет-источник (будущее)
│   ├── traveling_caravan.yaml # мобильный источник
│   ├── rain_system.yaml # природный источник
│   └── time_system.yaml # временной источник
├── archetypes/ # только preferences + goal_templates
│   ├── tavern_keeper.yaml
│   ├── maid.yaml
│   ├── guard.yaml
│   ├── merchant.yaml
│   ├── blacksmith.yaml
│   └── thief.yaml
├── simulation/
│   └── simulation_context.yaml # политики детализации (не свойство сущности)
├── secrets/
└── secret_contexts.yaml # 16 секретов с event-триггерами

backend/app/services/world_model/ # 6 независимых загрузчиков
├── ontology_registry.py # читает ontology/*.yaml
├── process_registry.py # читает ontology/processes.yaml
├── source_registry.py # читает sources/*.yaml
├── archetype_registry.py # читает archetypes/*.yaml
├── simulation_context.py # читает simulation/*.yaml + динамика
├── knowledge_registry.py # per-NPC NPCKnowledge (runtime)
├── world_graph.py # граф состояний мира
├── world_snapshot.py # текущее состояние мира
└── process_executor.py # запускает Process → EventDT0/SceneChange

backend/app/services/planner/
└── world_planner.py # поиск путей с эпистемическим фильтром

backend/app/models/world/
├── process.py # Process, ProcessSource, WorldStateDelta
├── affordance.py # AffordancePredicate (композиционный)
├── world_predicate.py # WorldPredicate для Planner'a
└── plan.py # Plan, PlanStep
```


7.1 Пример: ProcessSource (NPC)

```
# config/world/sources/blacksmith_orm.yaml
source_id: blacksmith_orm
# HET source_type – это просто источник
available_processes:
- process_id: item_repair
quality: 0.9
cost: { gold: 5, time_ticks: 5 }
requires_world_state: # требования живут здесь, не в Process
- { domain: 'infrastructure', attribute: 'metalwork', value: 'present' }
- { domain: 'resource', attribute: 'iron', value: 'available' }
requires_provider_state:
- { attribute: 'health', value: '>0' }
- { attribute: 'skill_blacksmith', value: '>0.5' }
availability:
hours: ['08:00', '18:00']
location_when_available: forge_orm

- process_id: information_acquisition # может делиться знанием
quality: 0.6
cost: { time_ticks: 2 }
topic_filter: [craft, metalwork, weapons]
availability:
hours: ['08:00', '22:00']
location_when_available: any

- process_id: social_bonding # может общаться
quality: 0.4
cost: { time_ticks: 3 }
```

7.2 Пример: ProcessSource (природный, без исполнителя)

```
# config/world/sources/rain_system.yaml
source_id: rain_system
# source_kind: environment – выводится автоматически из контекста
available_processes:
- process_id: rain_fills_river
quality: 1.0
cost: {} # бесплатный процесс
requires_world_state:
- { domain: 'weather', attribute: 'rain_intensity', value: '>0.5' }
availability:
condition: 'always_when_world_state_met'

- process_id: rain_extinguishes_fire
quality: 1.0
cost: {}
requires_world_state:
- { domain: 'environment', attribute: 'fire', value: 'present' }
- { domain: 'weather', attribute: 'rain_intensity', value: '>0.3' }
```

7.3 Пример: archetype — только preferences

```
# config/world/archetypes/merchant.yaml
archetype_id: merchant
long_term_preferences:
- ProfitSeeking { weight: 0.9 }
- RiskMinimization { weight: 0.6 }
- NetworkBuilding { weight: 0.5 }
- CustomerSearch { weight: 0.7 }
- ViolenceAvoidance { weight: 0.8 }

goal_templates:
- acquire_money
- find_customers
- maintain_reputation
- avoid_legal_trouble

risk_tolerance: 0.3
time_horizon: LONG # SHORT / MEDIUM / LONG
# НЕТ ссылок на capabilities, affordances, intents — только ценности
```

8. Архитектурные инварианты (расширение без правок)

Каждый пример демонстрирует, какие существующие компоненты **не пришлось менять**. Это и есть мера качества архитектуры.

Сценарий расширения	Что добавляется	Что НЕ меняется
Добавить нового NPC (напр. бард)	1 файл: sources/bard.yaml 1 файл: archetypes/bard.yaml 1 файл: config/npc/individuals/bard.json	Process registry, WorldGraph, Planner, NPCKnowledge, DecisionHub, Beliefs
Добавить новую локацию (напр. фестиваль)	1 файл: sources/festival_ground.yaml 1 файл: frontend/.../locations/festival_ground.json	ArcheType registry, Planner, Process registry. NPC автоматически начнут появляться
Добавить новый Process (напр. auction)	1 запись: ontology/processes.yaml + при необходимости — в available_processes источников, которые могут проводить аукцион	WorldGraph, Planner, NPCKnowledge, все архетипы, все существующие процессы
Добавить новый домен Affordances (напр. 'magic')	1 запись: ontology/domains.yaml - domain: magic attributes: [mana_level, spell_intensity, arcane_circle]	Все существующие sources, все архетипы, Planner. Старые предикаты не меняются
Добавить новый природный Process (напр. землетрясение)	1 запись: ontology/processes.yaml 1 файл: sources/earthquake_system.yaml	Все NPC, все локации, все архетипы. Planner автоматически учтёт при планировании
Сделать OFFSCREEN локацию FULL (играбельной)	Изменить 1 строку в simulation/simulation_detail_policy[forge_orm]: OFFSCREEN → FULL	Context registry, Process registry, WorldGraph, Planner, NPCKnowledge. Сами NPC не меняются
NPC сменил профессию (вор → наёмник)	RoleTransition.execute_transition() сбрасывает WorldGraph и Planner на следующем тике находит новые пути через archetype.mercenary	Context registry, Process registry, все остальные NPC
Раскрыть ложный слух (player узнаёт, что Тень лгал)	NPCKnowledge.mark_false(entry, revealed_by_player) player_belief_model.add_belief('shadow_is_liar')	WorldMemory (существующие поля), ADR-O-322, TruthState из ТЗ «Секретность»

9. Эпистемические гарантии (запрет телепатии)

Существующий ADR-O-322 уже запрещает телепатию на уровне восприятия. Новые слои (Planner, NPCKnowledge) добавляют эпистемический барьер на уровне **планирования**. Это два разных фронта:

Слой	Что запрещено	Как обеспечивается
Восприятие (ADR-O-322, уже есть)	NPC не может видеть/слышать то, что не проявилось в ObservableManifestationReceivedSignal на ObservableFactType.	ObservableManifestationReceivedSignal, ObservableFactType
Память (EventManager, уже есть)	NPC не может вспомнить то, чего не было в его EventMemory, скрыты hiddenEventMemoryManager, опубликованы publicEventMemoryManager.	EventMemory, hiddenEventMemoryManager, publicEventMemoryManager
Планирование (NPCKnowledge, НОВОЕ)	NPC не может планировать через ProcessSource, WorldPlanner не планирует через NPCKnowledge.	ProcessSource, WorldPlanner, NPCKnowledge
Знание о capabilities (NPCKnowledge, НОВОЕ)	NPC может знать source, но не знать его capabilities. KnowledgeEntity knows source, но не capabilities.	NPCKnowledge, KnowledgeEntity
Устаревание знания (world_revision, НОВОЕ)	Знание не «гниёт» само. Устаревает мир — NPCKnowledgeEntity не верифицирует world_revision, world_revision_verification.	NPCKnowledgeEntity, world_revision_verification
Ложные слухи (origin_chain, НОВОЕ)	Слухи имеют цепочку происхождения. Можно KnowledgeEntity, origin_chain и (origin_tick, ...) is_false=True помечать.	KnowledgeEntity, origin_chain, origin_tick

9.1 Существующий тест test_no_telepathy_in_ui_observation

УЖЕ ЕСТЬ: 00_CAUSAL_CONTRACT_v2.0.md:405. Тест уже описан в CAUSAL_CONTRACT. Расширяем его новыми assertions:

```
# существующее:
assert not ui_shows_secret_npc_doesnt_know

# новое (для Planner):
assert not npc_plans_through_unknown_source
assert npc_asks_for_information_when_no_known_path
assert npc_doesnt_use_capability_it_doesnt_know
assert rumor_chain_traceable_when_false
```

10. План внедрения (4 спринта, ~64 часа)

Спринт	Содержание	Часы
1. Ontology + Process layer	- models/world/{process,affordance,world_predicate,plan}.py - services/world_model/{ontology,process}_registry.py - config/world/ontology/{domains,processes}.yaml - Bridge AffordanceVector → AffordancePredicate - Тесты: each Process has valid requires/applies; AffordancePredicate.matches работает	~14 ч
2. ProcessSource registry + WorldGraph	- services/world_model/{source_registry,world_graph,world_snapshot}.py - config/world/sources/*.yaml (15 файлов: 6 NPC + 6 локаций + anvil/caravan/rain/time) - ProcessExecutor (мост к существующим EventDTO/SceneChange) - Тесты: WorldGraph.find_paths без эпистемики находит путь от сломанного меча к починенному	~16 ч
3. NPCKnowledge + Planner	- services/world_model/knowledge_registry.py - services/planner/world_planner.py - Расширение EventMemory: при information_acquisition — обновлять NPCKnowledge - Тесты: Lusya не планирует через orm_as_blacksmith, пока не узнает; после диалога — планирует - Тест: test_no_telepathy_in_planner (новый эпистемический тест)	~18 ч
4. Интеграция с DecisionHub + archetypes	- config/world/archetypes/*.yaml (6 архетипов с preferences + goal_templates) - GoalGenerator (преобразует preferences+needs → Goals → predicates) - DecisionHub для proactive intents делегирует в Planner; reactive остаётся как есть - simulation/simulation_context.yaml (детализация политик) - secrets/secret_contexts.yaml (16 секретов с event-триггерами) - Тесты: end-to-end — Lusya чинит меч через информацию от Орма	~16 ч
ИТОГО	Process-Oriented World Model как расширение ADR-O-322	~64 ч

10.1 Что НЕ делается в этом ТЗ

• **Не чинятся баги memory write path** (R1-R3 из отчёта №1) — это отдельная задача, без неё Planner будет работать, но не увидит эффекта на убеждениях NPC. • **Не реализуется TruthState / PlayerBeliefModel** из ТЗ «Секреты Люси» — это player-side, строится поверх NPC-side фундамента данного ТЗ. • **Не трогаются AffordanceVector** (motion_core.py) — физический слой остаётся. • **Не удаляются NodeRole** — используются в существующем RoleResolver'e, параллельно с AffordancePredicate. Удаляются позже, когда все consumers перейдут. • **Не меняется ADR-O-322** — 5 инвариантов остаются ненарушаемыми.

10.2 Зависимости от других работ

Спринт 1-2 можно делать параллельно с R1-R3 (memory write fixes). Спринт 3 (NPCKnowledge + Planner) требует, чтобы MemoryManager.apply работал — иначе information_acquisition process не обновит NPCKnowledge. Спринт 4 требует R1-R3 полностью починенными (иначе BeliefState не обновится от планов, и кристаллизация не сработает).

11. Итог

ПРОЦЕСС: **Принцип:** модернизация, не замена. Существующая Машина Эпистемологии (ADR-O-322) — фундамент. EventMemory, AffordanceVector, BeliefState, Manifestation, ObservationRelation — всё остаётся.

Добавляется 4 новых первоклассных сущности:

1. Process — абстрактная трансформация состояния мира (без исполнителя)
2. WorldGraph — граф состояний для поиска путей
3. NPCKnowledge — per-NPC субъективное знание (фильтр для Planner'a)
4. AffordancePredicate — композиционная онтология (domain, attribute, value)

Получается 7-слойный pipeline:

Need → Goal → RequiredWorldState → WorldGraph.find_paths()
→ Plan (with NPCKnowledge filter) → Process execution → Effect

Эпистемический барьер: NPC не знает — не планирует. Не знает capability — не использует. Слух ложный — цепочка происхождения восстанавливается. Мир изменился — знание помечается unverified.

Расширение без правок: добавить NPC/локацию/process/домен — 1-2 YAML файла. Существующий код не меняется. Это и есть масштабируемость от таверны к открытому миру.

Финальный тезис: NPC больше не «идёт в кузницу». NPC имеет Goal (починить меч), ищет путь в графе мира, фильтрует по тому, что знает, и если не знает — спрашивает знакомого. Кузнец, кузница, караван, найденный меч, украденный меч, магический ремонт — это всё разные маршруты по одному графу, а не разные ветви кода. Это и есть естественная масштабируемость симуляции.